RecallOps

Incident Response AI Agent

Product Requirements Document | Hackathon MVP, team working draft

Prepared 06 November 2026

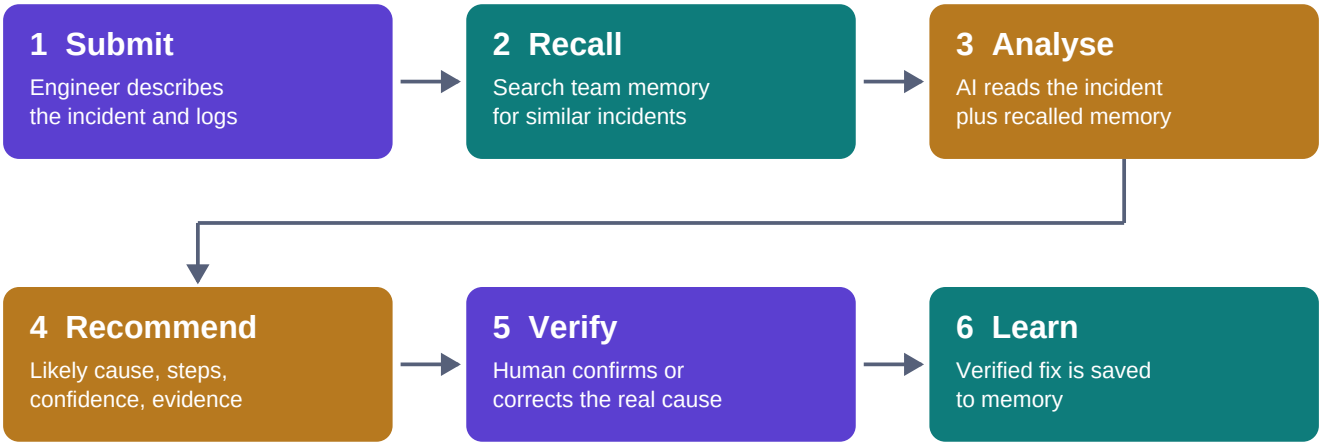
Recalled from memory AI hypothesis Human-verified

Document status	Hackathon MVP, team working draft
Prepared	06 November 2026
Target users	SREs, DevOps engineers, on-call engineers, engineering teams
Core technology	Hindsight memory + LLM reasoning (Groq) + FastAPI + Streamlit

Working principle: Incident > Memory Retrieval > AI Analysis > Human-verified Resolution > Learning

At a glance

RecallOps helps an engineer fix an incident faster by remembering how the team fixed similar ones before. The loop below is the whole product.



Step 6 feeds step 2: the next similar incident starts with what the team just learned.

Who it is for	What it does	What it never does
SREs, DevOps and on-call engineers who need to diagnose a service incident quickly.	Analyses an incident, recalls similar past incidents, suggests next steps, and learns from confirmed fixes.	Run commands, restart services or change production. It advises; a human decides.

What is inside

1 Executive summary
2 Problem statement
3 Proposed solution and key terms
4 Goals and non-goals
5 Users and use cases
6 Functional requirements
7 Non-functional requirements
8 Technical architecture
9 Screens
10 Demo data and scenario
11 Acceptance criteria
12 Team responsibilities
13 Risks and mitigations
14 Project structure
15 Implementation priority
16 Open decisions
17 Scope statement

1. Executive summary

RecallOps is an AI-assisted incident response agent for software and infrastructure teams. It helps engineers investigate service incidents by analysing submitted error details and logs, retrieving relevant experiences from Hindsight memory, and producing evidence-linked troubleshooting recommendations. When an engineer confirms the actual root cause and resolution, RecallOps stores that verified learning so it can improve responses to future, similar incidents.

The key differentiator is persistent operational memory. RecallOps should not behave like a generic chatbot that starts every incident from zero. It uses previous incident context, explains when a past incident is relevant, distinguishes historical evidence from a current hypothesis, and learns from human-confirmed outcomes.

> **How to read this document.** Teal means knowledge recalled from memory, amber means the AI's current hypothesis, and violet means a human has verified it. The product UI must keep these three apart.

2. Problem statement

During production incidents, engineers inspect logs, identify likely causes, search past tickets or chat, and decide which remediation steps are safe. This is slow and inconsistent when knowledge is scattered across people, documents and earlier sessions, and lessons are lost when team members change.

2.1 Core problem

> How might we help DevOps and SRE teams investigate incidents faster by making past incident knowledge searchable, reusable, and continuously improved through verified feedback?

2.2 Pain points

Pain point	What happens
Repeated troubleshooting	Similar failures may be investigated from scratch.
Fragmented knowledge	Root causes and fixes are spread across logs, tickets, notes and conversations.
Slow diagnosis	Engineers manually connect symptoms with previous incidents.
Inconsistent recommendations	Different responders suggest different investigation steps.
Knowledge loss	Confirmed fixes and lessons are not retained in a reusable form.

2.3 Intended outcome

Reduce repeated investigation effort and make incident response more consistent by combining LLM analysis with a persistent memory layer. RecallOps is an assistive decision-support tool. It does not autonomously execute production changes.

3. Proposed solution

Build an agent that accepts an incident description and log text, asks an LLM to analyse the symptoms, queries Hindsight for related past incidents, and returns a structured diagnosis with next steps. After a human resolves the incident, the responder submits the verified root cause and resolution, which RecallOps retains in Hindsight.

3.1 Core workflow

Stage	Expected behaviour
1. Submit incident	Engineer enters service name, error description, and optional log excerpt or context.
2. Retrieve memory	RecallOps queries Hindsight for semantically relevant prior incidents and lessons.
3. Analyse	The LLM considers the current incident with the retrieved memory and generates a structured assessment.
4. Recommend	The UI shows severity, likely cause, reasoning, recommended steps, confidence and any historical matches.
5. Human verifies	The engineer investigates and confirms or corrects the root cause and resolution.
6. Retain learning	RecallOps stores the verified outcome in Hindsight so later incidents benefit.

3.2 Why Hindsight is central

- **Retain:** save incident summaries and human-confirmed resolutions as persistent memory.
- **Recall:** retrieve relevant prior incidents when a new incident is analysed.
- **Reflect:** derive recurring patterns or lessons from accumulated memory, where the configured API supports it.
- **Demonstrate learning:** compare a first-time/generic response with a memory-informed one and show the retrieved evidence.

> **Rule:** the product must use actual Hindsight retrieval results in the demonstration. Do not hardcode a supposed historical match or present synthetic sample incidents as real production history.

3.3 Key terms in plain language

Term	What it means here
Hindsight	The memory service. RecallOps saves incident outcomes into it and searches it later.
Retain / Recall / Reflect	Save a lesson / find relevant past lessons / spot patterns across many lessons.
Groq (LLM)	The hosted AI model that reads the incident and memory and writes the analysis.
FastAPI	The backend. It validates requests and coordinates memory and AI. The UI only talks to it.
Streamlit	The simple web interface engineers use.
Verified outcome	A root cause and fix a human has explicitly confirmed. Only these count as learning.
Hypothesis	The AI's current guess. It is shown clearly, and never stored as fact.

4. Goals and non-goals

MVP goals	Explicit non-goals
• Accept an incident through a simple web interface. • Use Groq-hosted LLM inference for structured analysis. • Use Hindsight to retain and recall incident knowledge. • Show the link between retrieved memories and recommendations. • Let a human confirm or correct the root cause before memory is updated. • Offer a small insights view of retained incidents and themes, if time permits. • Deliver a reliable local demo with clear setup instructions.	• Live integration with Kubernetes, Prometheus, Grafana, PagerDuty or cloud monitoring. • Automatic execution of remediation commands, restarts, deployments or infrastructure changes. • Multi-agent orchestration, enterprise authentication or RBAC. • A full ticketing system or replacement for existing incident platforms. • Claims of production-grade reliability, security or compliance.

5. Target users and use cases

Primary users: SREs, DevOps engineers, on-call engineers and software engineers responsible for service reliability.

- As an on-call engineer, I want to submit an error and receive likely causes and next steps so I can begin investigating quickly.
- As an engineer, I want to see similar past incidents so I can reuse relevant troubleshooting knowledge.
- As an incident responder, I want to confirm or correct the suggested cause so incorrect AI assumptions are not stored as facts.
- As a team member, I want to inspect retained memories and recurring patterns so operational knowledge stays reusable.

6. Functional requirements

ID	Feature	Requirement	Priority
FR-01	Incident intake	User enters service name, incident/error description and optional log text.	Must
FR-02	Input validation	API validates required fields and returns understandable errors.	Must
FR-03	Memory recall	Backend queries the configured Hindsight bank for relevant previous incidents.	Must
FR-04	AI analysis	Backend sends the incident and retrieved context to the configured Groq model.	Must
FR-05	Structured result	Severity, likely root cause, explanation, recommended actions, confidence and historical match details.	Must
FR-06	Evidence transparency	UI distinguishes retrieved historical facts from the model's current hypotheses.	Must

ID	Feature	Requirement	Priority
FR-07	Human confirmation	User can submit the confirmed root cause and resolution after investigation.	Must
FR-08	Memory retention	Only human-confirmed outcomes are retained as verified learning.	Must
FR-09	Memory explorer	View or search retained memories, subject to Hindsight API capabilities.	Should
FR-10	Insights	Show recurring themes via Hindsight reflection or a clearly labelled fallback.	Could
FR-11	Health check	API exposes a health endpoint showing whether the service is running.	Must
FR-12	Demo comparison	Show a response without useful memory, then one informed by actual recalled memory.	Must

7. Non-functional requirements

Area	Requirement
Safety	Recommendations are advisory. No production action is executed automatically.
Traceability	Show retrieved memory references or identifiers when the API provides them.
Data integrity	Do not store an unverified AI guess as a confirmed root cause.
Reliability	Handle unavailable LLM/Hindsight services with a clear error state. Never silently fabricate results.
Security	Keep API keys in environment variables. Never commit .env or expose secrets in UI or logs.
Input handling	Treat pasted logs and incident text as untrusted data, not as instructions to the agent.
Usability	A new user can submit an incident and understand the result without reading implementation details.
Maintainability	Separate UI, API, agent logic, memory integration and data models.

8. Technical architecture

Layer	Role
Streamlit frontend	Incident form, analysis display, memory explorer, resolution confirmation.
FastAPI backend	REST endpoints, validation, orchestration, error handling.
Agent / LLM layer	Groq API for structured reasoning over the current incident and retrieved context.
Hindsight memory	Persistent incident knowledge: retain, recall and reflection.
Optional SQLite	Application-side audit/history or demo metadata. Not a replacement for Hindsight.

8.1 Suggested stack

Python; FastAPI + Uvicorn; Streamlit; Groq API (model chosen during implementation, candidates include openai/gpt-oss-120b and qwen/qwen3-32b); Hindsight Cloud or OSS; Pydantic; optional SQLite; Pytest; GitHub.

Implementation note: verify exact Hindsight SDK method names, request parameters, response fields and bank configuration against the current official documentation before finalising integration code.

8.2 Suggested API contract

Method	Endpoint	Purpose
GET	/health	Service health status.
POST	/analyze	Accept incident details, recall memory, return structured analysis.
POST	/resolve	Accept human-confirmed root cause and resolution, retain verified learning.
GET	/memories	List or search incident memories if supported.
GET	/insights	Return recurring themes or reflections if supported.

8.3 Suggested analysis response fields

`incident_id, severity (Low / Medium / High / Critical), likely_root_cause, explanation, recommended_actions (ordered list), confidence (model-estimated, not a guarantee), historical_matches (actual retrieved references, if any), memory_influence, caveats / missing_information`

9. User experience and screens

Screen	Contents
Analyze Incident	Form for service, error description and optional logs; submit button; loading and error states.
Analysis Result	Severity, likely cause, explanation, steps, confidence, retrieved historical matches and caveats.
Confirm Resolution	Fields for verified root cause and actual resolution; explicit confirmation before saving.
Memory Explorer	Retained incident memories and their verified outcomes where available.
System Insights	Recurring patterns or lessons from Hindsight reflection. Label demo/sample data clearly.

10. Demo data and scenario

Use clearly labelled **synthetic** incidents to seed the demo memory bank. They are fabricated for testing and must not be represented as real customer or production incidents.

ID	Service	Symptom	Synthetic root cause	Synthetic resolution
INC-001	Backend API	HTTP 500 errors	Database connection pool exhausted	Increase/configure pool capacity; investigate connection usage.
INC-002	Database	Connection timeouts	High database load / slow queries	Inspect query performance; optimise expensive queries.
INC-003	Payment Service	HTTP 502 errors	Upstream dependency unavailable	Verify upstream health; restore availability.

ID	Service	Symptom	Synthetic root cause	Synthetic resolution
INC-004	Auth Service	HTTP 401 errors	Expired authentication token	Refresh token; verify token lifecycle/configuration.
INC-005	Backend API	HTTP 503 errors	Memory exhaustion	Inspect memory usage, investigate leaks, adjust resources as appropriate.

10.1 Suggested live demo

1. Show RecallOps and explain the incident-response problem.
2. Submit a new backend incident with an error/log excerpt.
3. Show the initial analysis and the actual Hindsight memories retrieved.
4. Explain how a retrieved similar incident informs the recommended steps.
5. Show a no-match or memory-limited baseline, then compare it with the memory-informed result. Both paths must be genuine, not hardcoded.
6. Confirm a resolution as the human responder and retain it in Hindsight.
7. Submit a related follow-up incident and show the new learning being recalled.

11. MVP acceptance criteria

Done	Criterion
[ ]	A user can submit a valid incident through the UI.
[ ]	The API validates the request and returns a structured result.
[ ]	The application connects to the configured Hindsight bank.
[ ]	The analysis flow performs a real Hindsight recall and exposes the matches, or clearly reports none were found.
[ ]	The LLM response follows the agreed schema; invalid output is handled safely.
[ ]	The UI clearly labels suspected causes versus confirmed historical outcomes.
[ ]	A user can confirm a resolution and the application retains that verified outcome.
[ ]	A later related query can retrieve the retained learning from Hindsight.
[ ]	The demo uses actual retrieval results and labels synthetic seed data.
[ ]	Secrets are stored outside source control and the repository includes setup documentation.

12. Team responsibilities

Role	Ownership
Member 1: Backend/API	FastAPI setup, endpoints, request/response models, integration contract, API testing.
Member 2: Hindsight/Memories	Hindsight setup, bank configuration, retain/recall/reflect integration, synthetic seed data, memory verification.
Member 3: LLM/Agent	Groq integration, prompt design, structured output parsing, evidence-aware recommendations, fallback/error handling.

Role	Ownership
Member 4: UI/Delivery (if available)	Streamlit screens, API integration, README, demo script/video, presentation and submission content.

> **Coordination rule:** agree the API request/response schema before parallel implementation. Use separate Git branches, integrate frequently, and keep one shared list of environment-variable names.

13. Risks, assumptions and mitigations

Risk	Mitigation
Hindsight SDK/API differences	Verify current official docs early; isolate Hindsight calls in memory.py.
LLM output is malformed or overconfident	Strict schema, Pydantic validation, parse-error handling, confidence labelled as an estimate.
False historical match	Display actual retrieved evidence and similarity context; allow no-match; never invent incident IDs.
Unverified AI guess gets stored	Only the explicit human-confirmation workflow can mark an outcome verified.
External API/network failure	Show clear errors and preserve the user's input; do not fabricate success.
Time pressure	Prioritise the end-to-end workflow; defer dashboards and optional persistence.
Sensitive log content	Use synthetic/demo logs; avoid secrets and personal data; tell users to redact credentials.

14. Project structure

A clean split lets four people build in parallel. Items marked **[new]** go beyond the original PRD layout and are suggested to keep boundaries clear.

```
RecallOps/
|-- backend/
|   |-- main.py             FastAPI app and routes
|   |-- config.py           Settings from environment variables      [new]
|   |-- models.py           Pydantic request/response schemas
|   |-- agent.py            Groq prompt, structured output, parsing
|   |-- memory.py           Hindsight retain / recall / reflect
|   `-- database.py         Optional SQLite audit history
|-- frontend/
|   |-- app.py              Streamlit screens
|   `-- api_client.py       Calls the FastAPI backend only           [new]
|-- data/
|   `-- sample_incidents.json   Synthetic seed incidents (INC-001..005)
|-- scripts/
|   `-- seed_memory.py      Loads seed data into the Hindsight bank  [new]
|-- tests/
|   |-- test_agent.py       Prompt and schema parsing
|   |-- test_api.py         Endpoint validation and errors           [new]
|   `-- test_memory.py      Retain then recall round trip            [new]
|-- .env.example            Variable names only, no secrets
|-- .gitignore              Must list .env
|-- requirements.txt
`-- README.md               Setup, run steps, demo script
```

14.1 What each part owns

File	Responsibility	Owner
backend/main.py	FastAPI routes. Validates input, then calls memory and agent in order.	Member 1
backend/models.py	Pydantic schemas: the shared contract agreed before parallel work.	Member 1
backend/config.py	Loads settings and secrets from environment variables. [new]	Member 1
backend/memory.py	The only file that talks to Hindsight: retain, recall, reflect.	Member 2
backend/agent.py	The only file that talks to Groq: prompt, schema-checked output, error handling.	Member 3
backend/database.py	Optional SQLite audit history. Never replaces Hindsight.	Member 1
data/ and scripts/	Synthetic seed incidents and the script that loads them. [new script]	Member 2
frontend/	Streamlit screens, plus an API client that calls the backend only. [new client]	Member 4
tests/	Schema parsing, endpoint errors, retain-then-recall round trip.	Members 1, 2, 3
README.md	Setup, run steps and the demo script.	Member 4

14.2 How a request flows

Action	Path through the code
Analyse an incident	Streamlit > api_client > POST /analyze in main.py > validate (models.py) > memory.recall > agent.analyse > structured result back to the screen
Confirm a resolution	Streamlit > POST /resolve > validate > memory.retain, only after explicit human confirmation > saved as verified

14.3 Rules that keep it clean

- The frontend talks only to the FastAPI backend, never to Hindsight or Groq.
- Only memory.py imports the Hindsight client. Only agent.py imports the Groq client.
- models.py is the contract. Change it by team agreement, not alone.
- main.py coordinates; it holds no business logic of its own.
- Secrets are read in config.py from the environment and never committed.

14.4 Environment variables (suggested names)

Variable	Purpose
GROQ_API_KEY	Authenticates Groq requests.
GROQ_MODEL	Which Groq model to use (decided after testing).
HINDSIGHT_API_KEY	Authenticates Hindsight (if using Cloud).
HINDSIGHT_BASE_URL	Hindsight endpoint (Cloud or self-hosted).
HINDSIGHT_BANK_ID	The memory bank holding incident knowledge.
BACKEND_URL	Where the Streamlit app finds the API.

Confirm exact variable names and client settings against the current Hindsight documentation. List the names in .env.example and keep real values out of Git.

15. Implementation priority

Priority	Deliverable
P0 Foundation	Create repo, environment, env-var template, agree API schema, verify Hindsight and Groq credentials.
P0 Memory	Create/configure Hindsight bank, seed synthetic incidents, prove retain and recall end to end.
P0 Analysis	Implement incident analysis with Groq and structured output validation.
P0 API/UI	Connect FastAPI endpoints to Streamlit incident intake and results.
P0 Verified learning	Implement resolution confirmation and retain only verified outcomes.
P1 Demo polish	Memory explorer, insights, comparison view, README, tests, demo recording.

16. Open decisions for the team

- Hindsight Cloud or self-hosted OSS for the final demo?
- Which Groq-supported model gives the best quality and response time in testing?
- Will the MVP include SQLite audit history, or rely on Hindsight plus in-memory/session state?
- Which exact Hindsight operations and response fields does the selected SDK/API version support?
- What is the minimum set of log formats and incident fields supported in the demo?
- Who owns final integration, deployment/demo environment, video, article and social content?

17. Product scope statement

For the hackathon, RecallOps will deliver a working, human-in-the-loop incident response assistant that analyses a submitted incident, retrieves relevant prior experience from Hindsight, recommends evidence-aware next steps, and learns from confirmed resolutions. The success of the MVP is measured by a reliable end-to-end demonstration of memory improving incident investigation, not by the number of integrations or the amount of automation.